Universidad Tecnológica de El Salvador

FACULTAD DE INFORMÁTICA Y CIENCIAS APLICADAS

ESCUELA DE INFORMÁTICA

LENGUAJE UNIFICADO MODELADO (UML)

DOCENTE:

ING. JUAN JOSE SANTOS ESCOBAR

SECCIÓN: 01

TEMA DE PRESENTACIÓN:

LOGIFLOW360 ANÁLISIS DE ESTADOS

EQUIPO N: 06

INTEGRANTES:

ROL	CARNET	APELLIDOS	NOMBRES	PORCENTAJE DE TRABAJO
Líder de Arquitectura y Modelado UML	27-2589-2026	Alvarado Mejía	William Alberto	100%
Especialista en reglas y casos de borde	25-0554-2025	Lopez Cruz	Marcelo Daniel	100%
Ingeniero de Documentación y QA	27-1044-2026	Alas Minero	Luis Fernando	100%
Especialista en BD, Integración y Código	27-1555-2026	Portillo Ramos	Gerson Eliazar	100%

SAN SALVADOR, 22 DE OCTUBRE DE 2026.



I. Introducción *4*
Cómo leer las fichas 4
II. Entidad 01: OrdenDeCompra *5*
Diagrama de estados 5
Matriz de transición de estados 6
Análisis de casos de borde y resiliencia 9
III. Entidad 02: TransaccionDePago *10*
Diagrama de estados 10
Matriz de transición de estados 12
Análisis de casos de borde y resiliencia 16
IV. Entidad 03: PaqueteFisico *17*
Diagrama de estados 17
Matriz de transición de estados 18
Análisis de casos de borde y resiliencia 21
V. Entidad 04: VehiculoDeReparto *21*
Diagrama de estados 21
Matriz de transición de estados 22
Análisis de casos de borde y resiliencia 26
VI. Entidad 05: CasilleroInteligente *26*
Diagrama de estados 26
Matriz de transición de estados 27
Análisis de casos de borde y resiliencia 30
VII. Entidad 06: TicketDeSoporte *31*
Diagrama de estados 31
Matriz de transición de estados 32
Análisis de casos de borde y resiliencia 36
VIII. Entidad 07: SolicitudDeDevolucion *37*
Diagrama de estados 37
Matriz de transición de estados 39
Análisis de casos de borde y resiliencia 41
IX. Entidad 08: LoteDeInventario *42*
Diagrama de estados 42
Matriz de transición de estados 43
Análisis de casos de borde y resiliencia 46
X. Entidad 09: CuentaDeConductor *47*
Diagrama de estados 47
Matriz de transición de estados 47
Análisis de casos de borde y resiliencia 50

LO QUE HACES ¡IMPORTA!

XI. Entidad 10: DisputaBancaria 51

Diagrama de estados 51

Matriz de transición de estados 52

Análisis de casos de borde y resiliencia 54

XII. Diseño de Persistencia y Auditoría de Estados (Script SQL) 55

XIII. Propuesta de Implementación del Patrón State (Diagrama de Clases y Código Ilustrativo)..... 56

XIV. Lecciones Aprendidas y Conclusiones del Análisis 59

LO QUE HACES ¡IMPORTA!

I. Introducción

LogiFlow 360° es una plataforma logística de comercio electrónico que acompaña cada compra desde el carrito y el pago hasta la entrega, las devoluciones y las disputas bancarias. En ese recorrido intervienen diez entidades (órdenes, pagos, paquetes, vehículos, casilleros inteligentes, tickets, devoluciones, lotes, conductores y disputas), cada una con un ciclo de vida propio.

El modelado dinámico con máquinas de estados UML 2.5 aporta estabilidad porque define, para cada entidad, qué estados puede tener y qué evento, condición y acción permite pasar de uno a otro. Lo que no está dibujado no puede ocurrir, así que el modelo previene inconsistencias típicas: inventario reservado sin cobro, pagos reintentados sin límite, paquetes que saltan etapas de custodia o entidades detenidas sin salida.

Este informe presenta un diagrama por entidad, construido en StarUML (archivo `LogiFlow360_StateModels.mdj`), con su matriz de transición y su análisis de resiliencia. Todas las transiciones siguen la sintaxis `Evento [Guarda] / Acción()`.

Cómo leer las fichas

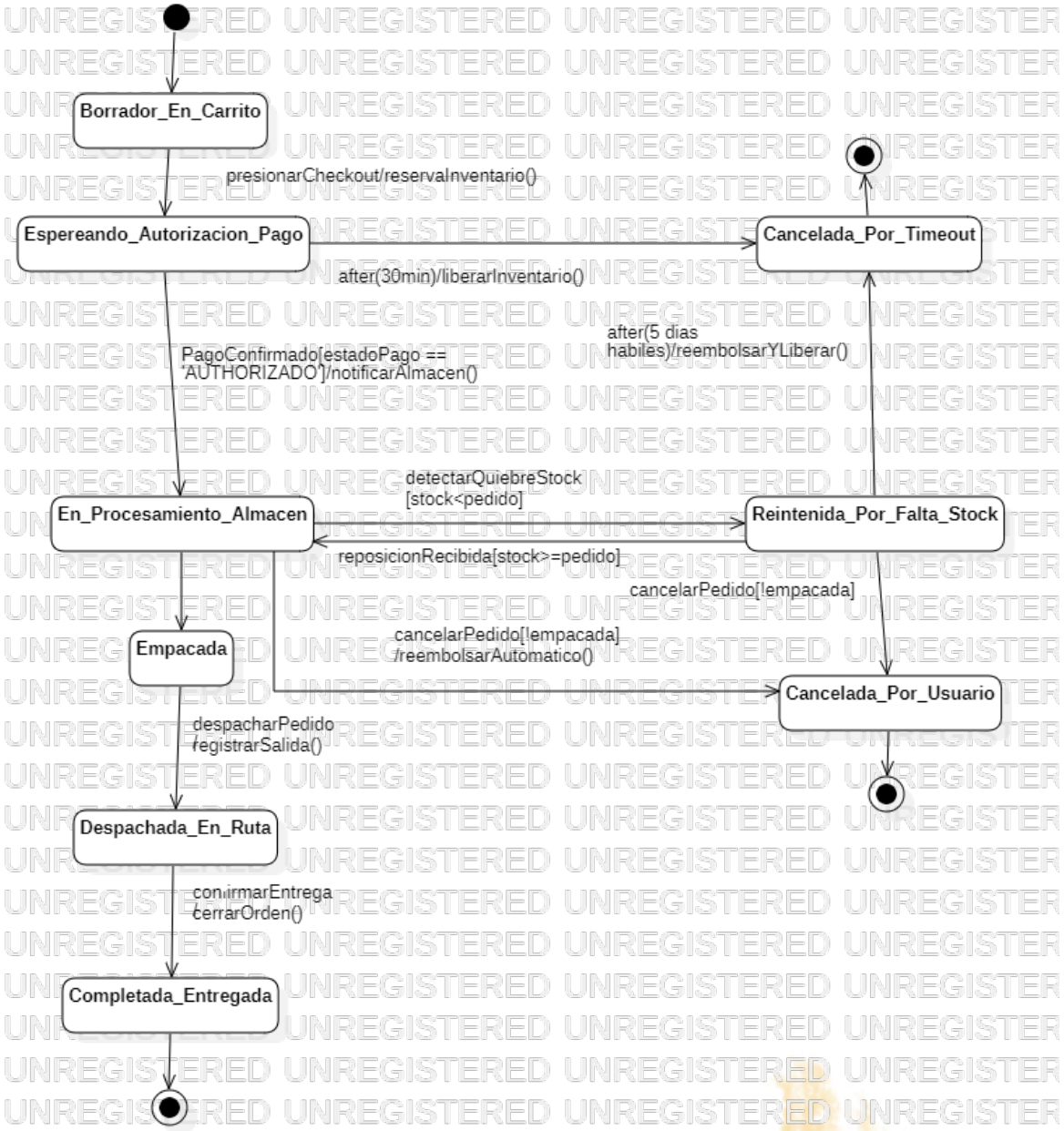
- **Diagrama:** captura PNG de alta resolución exportada desde StarUML, con fondo claro y sin distorsión.
- **Matriz de transición:** una fila por transición, con Estado origen, Evento disparador, Condición de guarda, Efecto o acción, Estado destino y Justificación. – indica que no aplica; `Inicio` es el pseudoestado inicial, `Final` el estado final y `Decisión` un pseudoestado Choice.
- **Análisis de resiliencia:** cómo maneja el diagrama las caídas de red, los timeouts, las cancelaciones abruptas y los reintentos. Donde el modelo no define un mecanismo, la ficha lo indica.
- Los eventos, guardas y acciones de las Entidades 03 a 10 son propuestas derivadas del enunciado; deben coincidir con los diagramas dibujados en StarUML.

II. Entidad 01: OrdenDeCompra

Objeto representado: entidad agregadora de la compra del cliente. Cubre el ciclo transaccional y comercial desde el carrito hasta la entrega o la cancelación.

Diagrama de estados

Figura 1. Máquina de estados de `OrdenDeCompra`.



LO QUE HACES ¡IMPORTA!

Matriz de transición de estados

Estado origen	Evento disparador	Condición de guarda	Efecto / acción	Estado destino	Justificación arquitectónica
Inicio	—	—	—	`Borrador_En_Carrito`	La orden nace como borrador mientras el cliente arma el carrito.
`Borrador_En_Carrito`	`presionarCheckout`	—	`reservarInventario()`	`Esperando_Autorizacion_Pago`	El checkout congela el pedido y reserva el stock para que nadie más lo tome durante el pago.
`Esperando_Autorizacion_Pago`	`pagoConfirmado`	`[estadoPago == 'AUTORIZADO']`	`notificarAlmacen()`	`En_Procesamiento_Almacen`	Solo un pago autorizado habilita preparar la mercadería; evita despachar sin cobro.

LO QUE HACES ¡IMPORTA!

Estado origen	Evento disparador	Condición de guarda	Efecto / acción	Estado destino	Justificación arquitectónica
Esperando_Autorizacion_Pago	after(30 min)	—	liberarInventario()	Cancelada_Por_Timeout	Sin pago en 30 minutos se libera el stock reservado.
En_Procesamiento_Almacen	confirmarEmpaque	—	generarEtiqueta()	Empacada	El almacén cierra la preparación y deja el bulto listo para transporte.
En_Procesamiento_Almacen	detectarQuiebreStock	[stockDisponible < cantidadPedida]	notificarCliente()	Retenida_Por_Falta_Stock	Un quiebre no cancela de inmediato: la orden espera reposición.
Retenida_Por_Falta_Stock	reposicionRecibida	[stockDisponible >= cantidadPedida]	—	En_Procesamiento_Almacen	Con stock repuesto la orden retoma su preparación.

Estado origen	Evento disparador	Condición de guarda	Efecto / acción	Estado destino	Justificación arquitectónica
Retenida_Por_Falta_Stock	after(5 dias habiles)	—	reembolsarYLiberar()	Cancelada_Por_Timeout	La espera tiene un máximo de 5 días hábiles; luego la orden se auto-cancela con reembolso.
En_Procesamiento_Almacen	cancelarPedido	[!empacada]	reembolsarAutomatico()	Cancelada_Por_Usuario	El cliente cancela mientras el pedido no esté empacado; el reembolso es automático.
Retenida_Por_Falta_Stock	cancelarPedido	[!empacada]	reembolsarAutomatico()	Cancelada_Por_Usuario	El mismo derecho de cancelación se mantiene durante la espera de reposición.

Estado origen	Evento disparador	Condición de guarda	Efecto / acción	Estado destino	Justificación arquitectónica
`Empacada`	`despacharPedido`	—	`registrarSalida()`	`Despachada_En_Ruta`	La salida registra la entrega de la custodia al transporte.
`Despachada_En_Ruta`	`confirmarEntrega`	—	`cerrarOrden()`	`Completada_Entregada`	La entrega confirmada cierra el ciclo comercial de la orden.
`Completada_Entregada`, `Cancelada_Por_Timeout`, `Cancelada_Por_Usuario`	—	—	—	Final	Fin del ciclo de vida: ningún evento posterior se acepta.

Análisis de casos de borde y resiliencia

Escenario	Cómo lo maneja el diagrama
Caídas de red	No hay un estado de red. Si la confirmación del pago se pierde, `after(30 min)` evita que la orden

LO QUE HACES ¡IMPORTA!

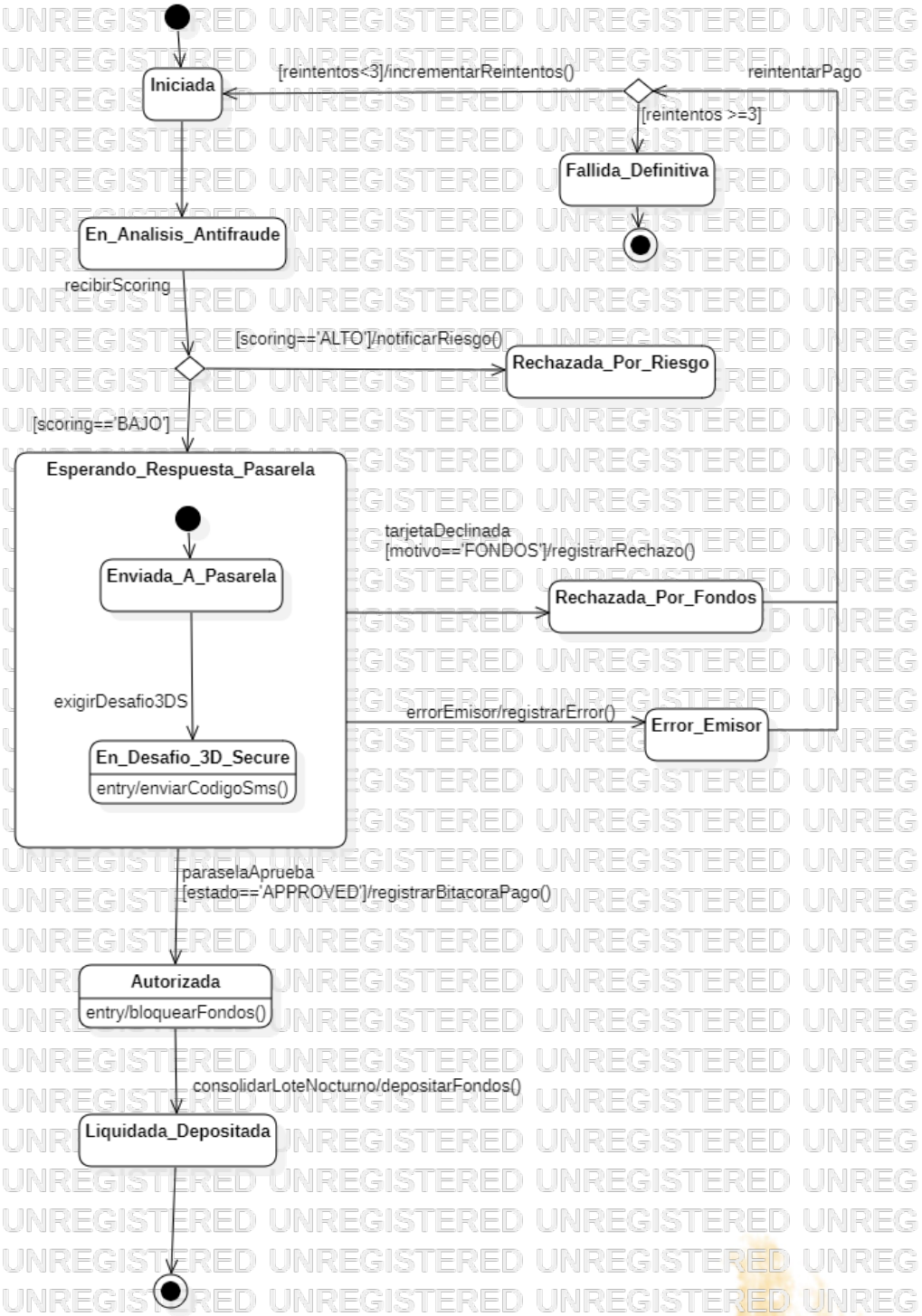
Escenario	Cómo lo maneja el diagrama
	espere indefinidamente: pasa a `Cancelada_Por_Timeout` y libera el inventario.
Timeouts	`after(30 min)` en `Esperando_Autorizacion_Pago` cancela la orden sin pago. `after(5 dias habiles)` en `Retenida_Por_Falta_Stock` la cancela con reembolso.
Cancelaciones abruptas	`cancelarPedido [!empacada]` desde `En_Procesamiento_Almacen` o `Retenida_Por_Falta_Stock` lleva a `Cancelada_Por_Usuario` con reembolso automático. No existe desde `Empacada` ni `Despachada_En_Ruta`.
Reintentos	La orden no reintenta pagos; eso lo hace `TransaccionDePago`. El único retorno es de `Retenida_Por_Falta_Stock` a `En_Procesamiento_Almacen` cuando llega la reposición.
Quiebre de stock	`detectarQuiebreStock` retiene la orden en lugar de cancelarla y espera reposición hasta 5 días hábiles.

III. Entidad 02: TransaccionDePago

Objeto representado: registro financiero individual con la pasarela bancaria (Stripe, PayPal o banco emisor). Cubre el análisis antifraude, la autorización y la liquidación del pago.

Diagrama de estados

Figura 2. Máquina de estados de `TransaccionDePago`, con el estado compuesto abierto.



UNREGISTERED UNREGISTERED UNREGISTERED UNREG

Matriz de transición de estados

Estado origen	Evento disparador	Condición de guarda	Efecto / acción	Estado destino	Justificación arquitectónica
Inicio	—	—	—	`Iniciada`	La transacción nace al registrarse el intento de pago.
`Iniciada`	`solicitarAnalisis`	—	`enviarDatosAntifraude()`	`En_Analisis_Antifraude`	Todo pago pasa por antifraude antes de llegar a la pasarela.
`En_Analisis_Antifraude`	`recibirScoring`	—	—	Decisión C1	El resultado del scoring se evalúa en un punto de decisión explícito.
Decisión C1	—	`[scoring == 'ALTO']`	`notificarRiesgo()`	`Rechazada_Por_Riesgo`	Un riesgo alto corta el flujo sin consultar a la pasarela.

Estado origen	Evento disparador	Condición de guarda	Efecto / acción	Estado destino	Justificación arquitectónica
Decisión C1	—	`[scoring == 'BAJO']`	—	`Esperando_Respuesta_Pasarela`	Con riesgo bajo se puede consultar a la pasarela.
Inicio del compuesto	—	—	—	`Enviada_A_Pasarela`	Al entrar al compuesto, la solicitud viaja a la pasarela.
`Enviada_A_Pasarela`	`exigirDesafio3DS`	—	—	`En_Desafio_3D_Secure`	Si el emisor exige autenticación reforzada, el usuario valida con un código SMS.
`Esperando_Respuesta_Pasarela` (desde cualquier subestado)	`pasarelaAprueba`	`[estado == 'APPROVED']`	`registrarBitacoraPago()`	`Autorizada`	Solo la aprobación explícita de la pasarela autoriza el pago, y queda

LO QUE HACES ¡IMPORTA!

Estado origen	Evento disparador	Condición de guarda	Efecto / acción	Estado destino	Justificación arquitectónica
					bitácora para auditoría.
Esperando_Respuesta_Pasarela (desde cualquier subestado)	tarjetaDeclinada	[motivo == 'FONDOS']	registrarRechazo()	Rechazada_Por_Fondos	Distingue el rechazo por saldo insuficiente.
Esperando_Respuesta_Pasarela (desde cualquier subestado)	errorEmisor	—	registrarError()	Error_Emisor	Separa una falla del banco emisor de un rechazo del cliente.
Rechazada_Por_Fondos	reintentarPago	—	—	Decisión C2	El reintento manual pasa por un control de límite.
Error_Emisor	reintentarPago	—	—	Decisión C2	Igual que el rechazo por fondos.
Decisión C2	—	[reintentos < 3]	incrementarReintentos()	Iniciada	Se permite reintentar

Estado origen	Evento disparador	Condición de guarda	Efecto / acción	Estado destino	Justificación arquitectónica
					mientras no se agoten los 3 intentos.
Decisión C2	—	`[reintentos >= 3]`	—	`Fallida_Definitiva`	Superado el límite, la falla es definitiva.
`Autorizada`	`consolidarLoteNocturno`	—	`depositarFondos()`	`Liquidada_Depositada`	El lote bancario nocturno liquida y deposita los fondos.
`Liquidada_Depositada`, `Rechazada_Por_Riesgo`, `Fallida_Definitiva`	—	—	—	Final	Fin del ciclo de vida de la transacción.

Acciones internas: `En_Analisis_Antifraude` ejecuta `do / calcularScoringRiesgo()`, `En_Desafio_3D_Secure` ejecuta `entry / enviarCodigoSms()` y `Autorizada` ejecuta `entry / bloquearFondos()`. `Esperando_Respuesta_Pasarela` es un estado compuesto con los subestados `Enviada_A_Pasarela` y `En_Desafio_3D_Secure`.

Análisis de casos de borde y resiliencia

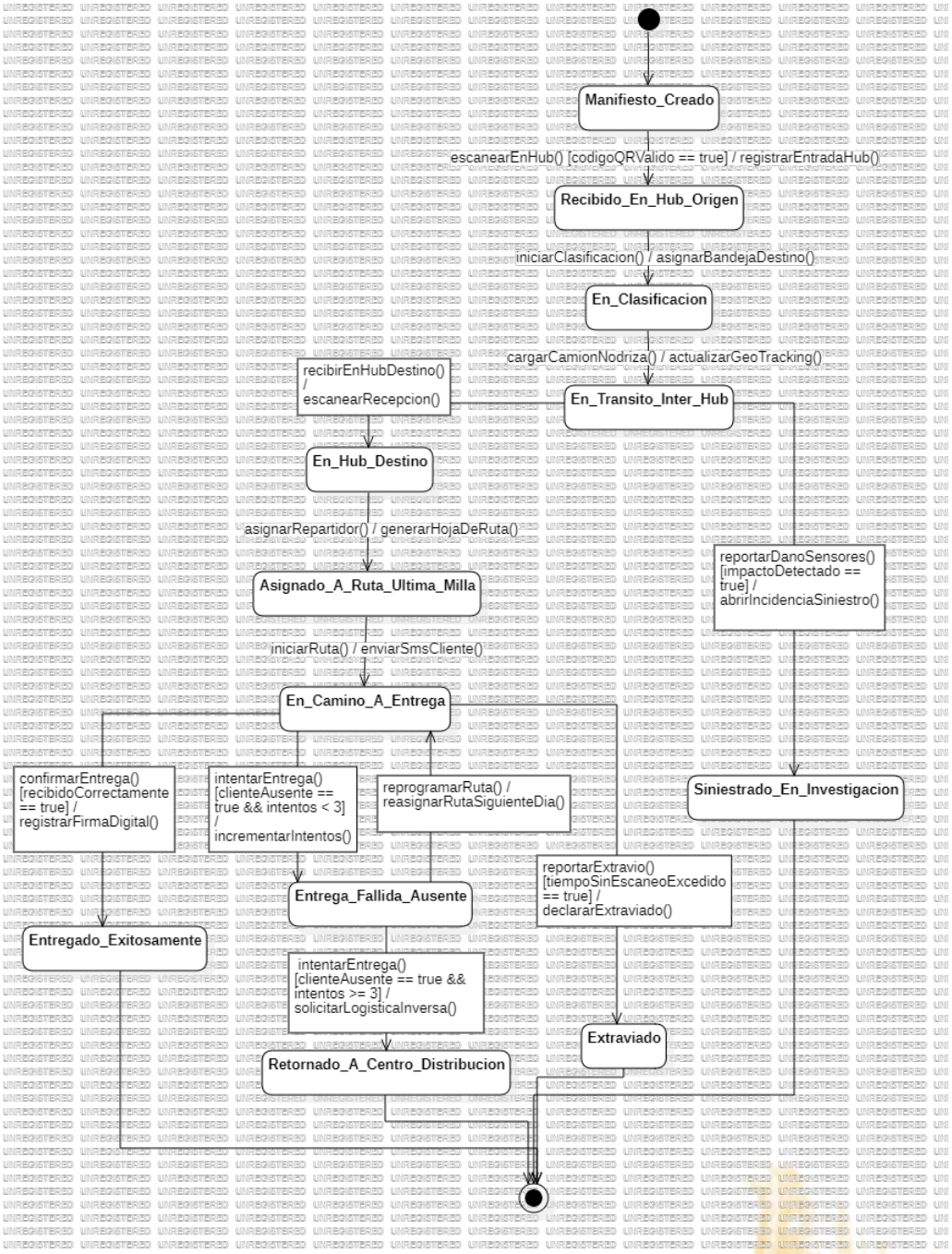
Escenario	Cómo lo maneja el diagrama
Caídas de red	Una falla de comunicación con el banco se modela como `errorEmisor`, que lleva a `Error_Emisor` y admite reintento. No hay estado ni temporizador para una pasarela que no responde (ver observaciones).
Timeouts	La entidad no define temporizadores; la liquidación ocurre cuando llega `consolidarLoteNocturno`. Tampoco hay un tiempo máximo para `Enviada_A_Pasarela` ni para `En_Desafio_3D_Secure` (ver observaciones).
Cancelaciones abruptas	Un scoring alto corta el pago de inmediato (`Rechazada_Por_Riesgo`) sin consultar a la pasarela. La cancelación del cliente no está modelada aquí: depende de la orden (Entidad 01).
Reintentos	`reintentarPago` pasa por la decisión C2: con `reintentos < 3` vuelve a `Iniciada` y con 3 o más termina en `Fallida_Definitiva`.
Fallo de tarjeta	`tarjetaDeclinada [motivo == 'FONDOS']` lleva a `Rechazada_Por_Fondos` y `errorEmisor` lleva a `Error_Emisor`. Ambos admiten reintento.
Autenticación 3D Secure	Si la pasarela la exige, el pago pasa a `En_Desafio_3D_Secure`; solo la aprobación posterior lleva a `Autorizada`.

IV. Entidad 03: PaqueteFisico

Objeto representado: bulto físico etiquetado con código de barras o QR. Modela la cadena de custodia y el tracking logístico desde el hub de origen hasta el destino.

Diagrama de estados

Figura 3. Máquina de estados de `PaqueteFisico`.



LO QUE HACES ¡IMPORTA!

Matriz de transición de estados

Estado origen	Evento disparador	Condición de guarda	Efecto / acción	Estado destino	Justificación arquitectónica
Inicio	—	—	—	`Manifiesto_Creado`	El paquete nace cuando se genera su manifiesto.
`Manifiesto_Creado`	`escanearEnHubOrigen`	—	`registrarRecepcion()`	`Recibido_En_Hub_Origen`	El primer escaneo confirma que el bulto entró a la cadena de custodia.
`Recibido_En_Hub_Origen`	`iniciarClasificacion`	—	—	`En_Clasificacion`	El bulto se ordena según su destino.
`En_Clasificacion`	`despacharEntreHubs`	—	`asignarCamionNodriza()`	`En_Transito_Inter_Hub`	La clasificación termina con la carga al camión nodriza.
`En_Transito_Inter_Hub`	`llegarAHubDestino`	—	`registrarEscaneoHub()`	`En_Hub_Destino`	El escaneo de llegada actualiza la custodia.
`En_Hub_Destino`	`asignarRuta`	—	`vincularVehiculo()`	`Asignado_A_Ruta_Ultima_Milla`	Se vincula el bulto con un vehículo y una

LO QUE HACES ¡IMPORTA!

Estado origen	Evento disparador	Condición de guarda	Efecto / acción	Estado destino	Justificación arquitectónica
					ruta de entrega.
Asignado_A_Ruta_Ultima_Milla	iniciarEntrega	—	notificarCliente()	En_Camino_A_Entrega	La salida del repartidor activa el aviso al cliente.
En_Camino_A_Entrega	confirmarEntrega	—	registrarEvidenciaEntrega()	Entregado_Exitosamente	La entrega queda respaldada con evidencia, útil ante una disputa.
En_Camino_A_Entrega	reportarAusencia	—	incrementarIntentos()	Entrega_Fallida_Ausente	Nadie recibe el paquete: se cuenta el intento fallido.
Entrega_Fallida_Ausente	evaluarIntentos	—	—	Decisión C1	El contador de intentos (1, 2 o 3) se evalúa en un punto de decisión.
Decisión C1	—	[intentos < 3]	reprogramarEntrega()	Asignado_A_Ruta_Ultima_Milla	Se permite otro intento de entrega.

Estado origen	Evento disparador	Condición de guarda	Efecto / acción	Estado destino	Justificación arquitectónica
Decisión C1	—	[intentos >= 3]	notificarRetorno()	Retornado_A_Centro_Distribucion	Al tercer intento fallido el paquete regresa al centro de distribución.
En_Transito_Inter_Hub, En_Camino_A_Entrega	detectarImpacto	[impacto > umbral]	abrirInvestigacion()	Siniestrado_En_Investigacion	Los sensores de impacto detectan daño durante el traslado.
En_Transito_Inter_Hub, En_Camino_A_Entrega	reportarExtravio	—	abrirInvestigacion()	Extraviado	Un reporte del transportista declara el bulto perdido.
Entregado_Exitosamente, Retornado_A_Centro_Distribucion, Siniestrado_En_Inves	—	—	—	Final	Fin del ciclo de custodia de este paquete.

Estado origen	Evento disparador	Condición de guarda	Efecto / acción	Estado destino	Justificación arquitectónica
`tigacion`, `Extraviado`					

Análisis de casos de borde y resiliencia

Escenario	Cómo lo maneja el diagrama
Caídas de red	Los avances dependen de escaneos. Si el escáner o la red fallan, el evento llega tarde y el paquete permanece en su estado actual: ninguna transición salta etapas de custodia.
Timeouts	El diagrama no define temporizadores. Un paquete que deja de escanearse se detecta con `reportarExtravio`, enviado por el transportista.
Cancelaciones abruptas	Una interrupción no prevista se modela como `Siniestrado_En_Investigacion` (impacto) o `Extraviado` (reporte del transportista). La cancelación del envío por el cliente no está modelada.
Reintentos	`Entrega_Fallida_Ausente` cuenta los intentos y la decisión C1 permite reprogramar hasta el tercero; luego el paquete vuelve al centro de distribución.

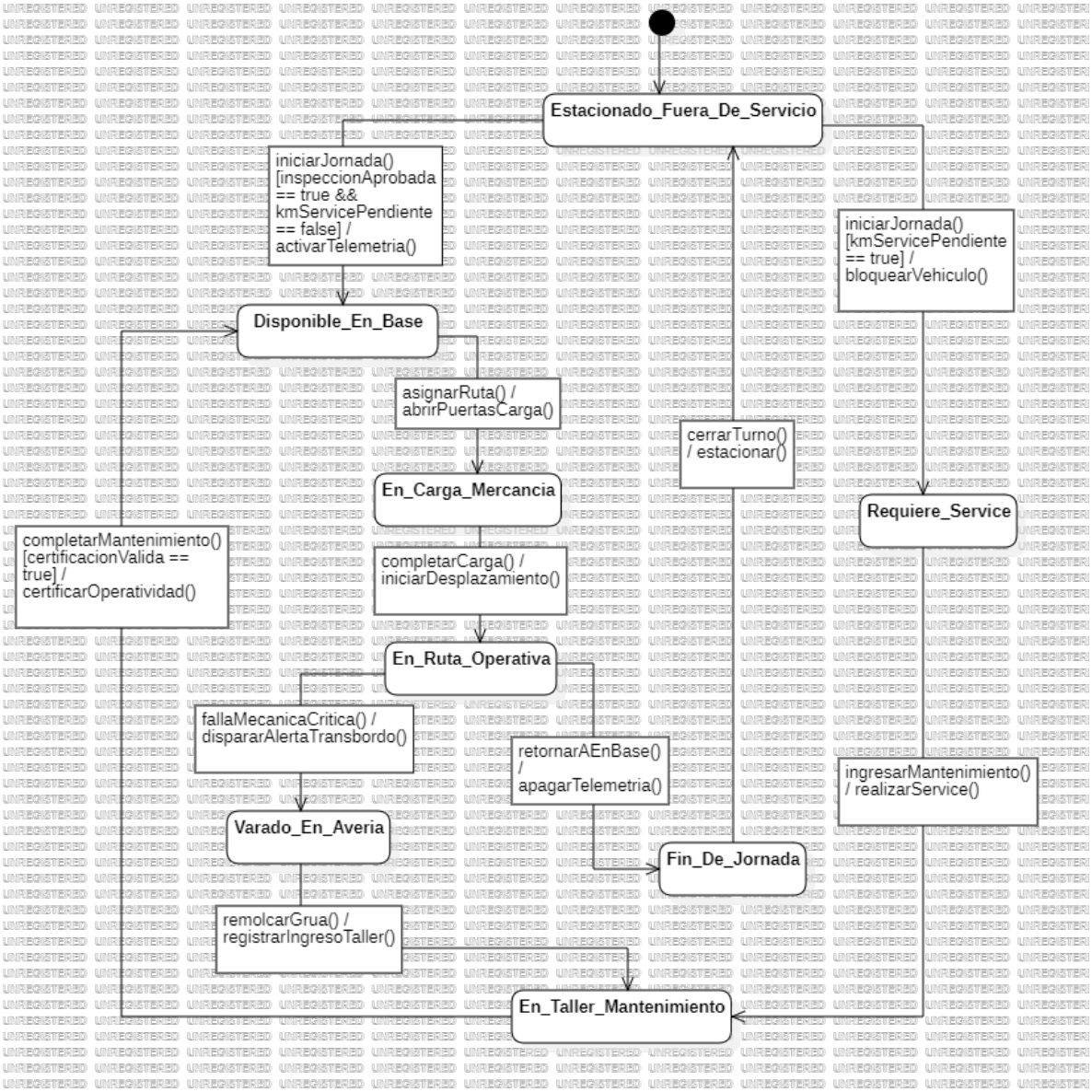
V. Entidad 04: VehiculoDeReparto

Objeto representado: activo físico de la flota (camioneta o motocicleta de reparto). Modela la jornada, la telemetría y el mantenimiento de la unidad.

Diagrama de estados

Figura 4. Máquina de estados de `VehiculoDeReparto`.

LO QUE HACES ¡IMPORTA!



Matriz de transición de estados

Estado origen	Evento disparador	Condición de guarda	Efecto / acción	Estado destino	Justificación arquitectónica
Inicio	—	—	—	`Estacionado_Fuera_De_Servicio`	El vehículo pasa la

LO QUE HACES ¡IMPORTA!

Estado origen	Evento disparador	Condición de guarda	Efecto / acción	Estado destino	Justificación arquitectónica
					noche fuera de servicio.
Estacionado_Fuera_De_Servicio	iniciarJornada	[inspeccionTecnica == 'APROBADA' && mantenimientoAlDia]	registrarInicioJornada()	Disponible_En_Base	Solo una unidad inspeccionada y con mantenimiento al día sale a operar.
Estacionado_Fuera_De_Servicio	iniciarJornada	[!mantenimientoAlDia]	bloquearUnidad()	Requiere_Service	La telemetría detecta mantenimiento preventivo vencido por kilometraje; la unidad no puede pasar a Disponible.
Disponible_En_Base	asignarRuta	—	registrarRuta()	En_Carga_Mercancia	El despachador asigna la ruta y se

Estado origen	Evento disparador	Condición de guarda	Efecto / acción	Estado destino	Justificación arquitectónica
					carga la mercancía.
En_Carga_Mercancia	salirARuta	—	iniciarTelemetria()	En_Ruta_Operativa	La salida a la calle activa el seguimiento de la unidad.
En_Ruta_Operativa	finalizarEntregas	—	registrarRetorno()	Fin_De_Jornada	Cierra la jornada de reparto.
En_Ruta_Operativa	fallaMecanicaCritica	—	alertarTransbordo()	Varado_En_Averia	Una avería crítica dispara la alerta para transbordar los paquetes a otro vehículo.
Varado_En_Averia	remolcarUnidad	—	registrarRemolque()	En_Taller_Mantenimiento	La unidad remolcada ingresa al taller.

Estado origen	Evento disparador	Condición de guarda	Efecto / acción	Estado destino	Justificación arquitectónica
En_Taller_Mantenimiento	completarReparacion	—	registrarMantenimiento()	Estacionado_Fuera_De_Servicio	Tras la reparación la unidad vuelve al ciclo y repite la inspección.
Requiere_Service	enviarATaller	—	programarService()	En_Taller_Mantenimiento	Solo el mantenimiento libera una unidad bloqueada.
Fin_De_Jornada	estacionarUnidad	—	descargarTelemetria()	Estacionado_Fuera_De_Servicio	La unidad regresa a su estado nocturno y el ciclo se repite.

Acciones internas: `En_Ruta_Operativa` ejecuta `do / monitorearUbicacionGPS()`. El ciclo es continuo, por lo que el diagrama no tiene

LO QUE HACES ¡IMPORTA!

estado final.

Análisis de casos de borde y resiliencia

Escenario	Cómo lo maneja el diagrama
Caídas de red	Si la telemetría se interrumpe, el estado no cambia: la unidad sigue en `En_Ruta_Operativa` hasta que llegue un evento. El modelo no define un estado por pérdida de señal.
Timeouts	No hay temporizadores. La jornada termina con `finalizarEntregas`, no por tiempo.
Cancelaciones abruptas	Una avería en ruta interrumpe la jornada: `fallaMecanicaCritica` lleva a `Varado_En_Averia` y dispara la alerta de transbordo de paquetes.
Reintentos	Si la inspección no se aprueba, la unidad permanece en `Estacionado_Fuera_De_Servicio`; si el mantenimiento está vencido, queda en `Requiere_Service` hasta pasar por el taller.

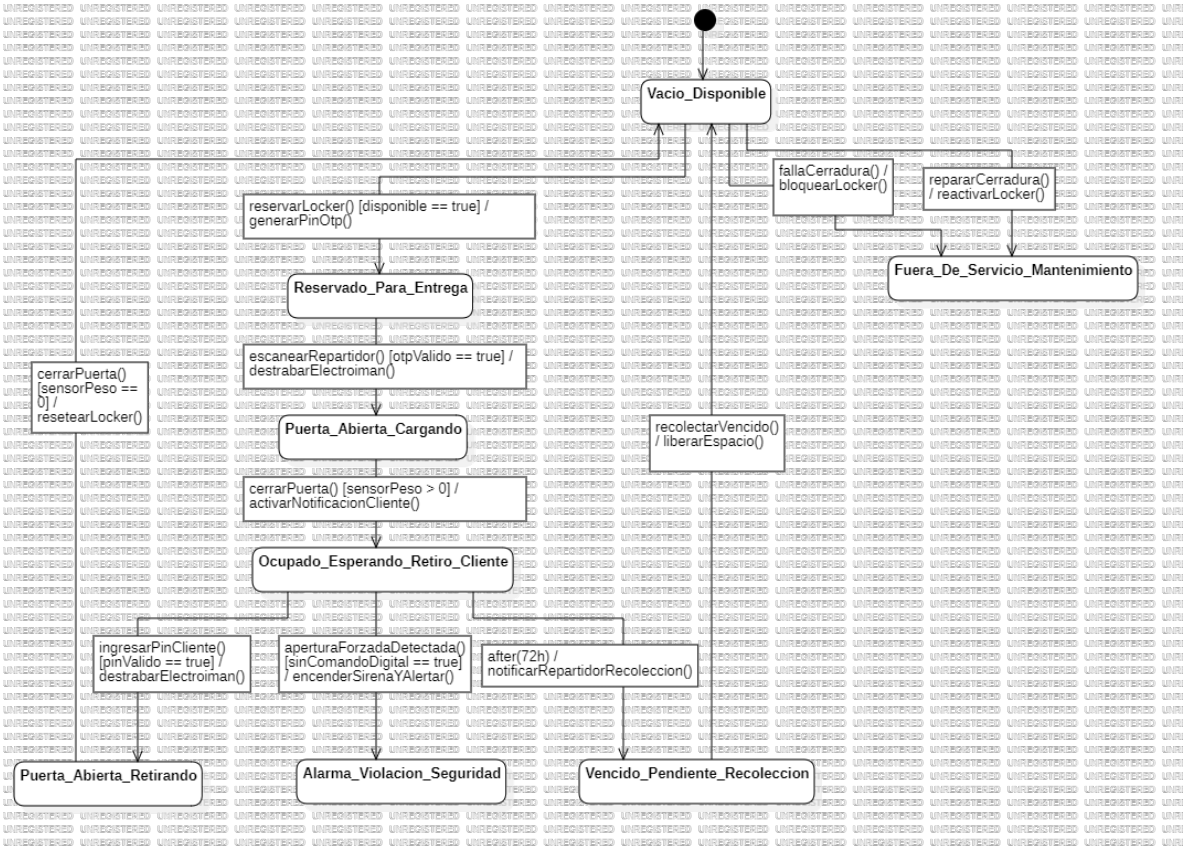
VI. Entidad 05: CasilleroInteligente

Objeto representado: puerta individual electromecánica de un casillero público autónomo (IoT). Controla el compartimiento físico, los códigos temporales de retiro y el estado del hardware.

Diagrama de estados

Figura 5. Máquina de estados de `CasilleroInteligente`.

LO QUE HACES ¡IMPORTA!



Matriz de transición de estados

Estado origen	Evento disparador	Condición de guarda	Efecto / acción	Estado destino	Justificación arquitectónica
Inicio	—	—	—	`Vacio_Disponible`	El compartimiento nace libre.
`Vacio_Disponible`	`reservarParaPaquete`	—	`asignarPaquete()`	`Reservado_Para_Entrega`	El sistema reserva la puerta para un paquete entrante.

LO QUE HACES ¡IMPORTA!

Estado origen	Evento disparador	Condición de guarda	Efecto / acción	Estado destino	Justificación arquitectónica
Reservado_Para_Entrega	escanearOrdenRepartidor	[codigoOrdenValido]	abrirCerradura()	Puerta_Abierta_Cargando	Solo el repartidor con la orden correcta abre la puerta.
Puerta_Abierta_Cargando	cerrarPuerta	[pesoSensor > 0]	trabarCerradura(), generarPinTemporal()	Ocupado_Esperando_Retiro_Cliente	El sensor de peso confirma que el paquete quedó dentro y se genera el PIN o QR temporal.
Ocupado_Esperando_Retiro_Cliente	ingresarPinOQr	[pinValido]	abrirCerradura()	Puerta_Abierta_Retirando	El PIN o QR temporal autentica al cliente.
Puerta_Abierta_Retirando	cerrarPuerta	[pesoSensor == 0]	trabarCerradura()	Vacio_Disponible	El peso cero confirma el retiro y libera el

Estado origen	Evento disparador	Condición de guarda	Efecto / acción	Estado destino	Justificación arquitectónica
					compartimiento.
Ocupado_Esperando_Retiro_Cliente	after(72h)	—	notificarVencimiento()	Vencido_Pendiente_Recoleccion	Pasadas 72 horas sin retiro, el paquete se recoge para devolverlo al hub.
Vencido_Pendiente_Recoleccion	recolectarPaquete	[pesoSensor == 0]	registrarDevolucionAlHub()	Vacio_Disponible	El repartidor retira el paquete y la puerta queda libre.
Vacio_Disponible, Reservado_Para_Entrega, Ocupado_Esperando_Retiro_Cliente, Vencido_Pendiente_Recoleccion	detectarAperturaForzada	[!comandoDigitalPrevio]	activarAlarma()	Alarma_Violacion_Seguridad	Una apertura sin comando digital se trata como violación de seguridad.

Estado origen	Evento disparador	Condición de guarda	Efecto / acción	Estado destino	Justificación arquitectónica
`Puerta_Abierta_Cargando`, `Puerta_Abierta_Retirando`	`fallaCerradura`	`[cerraduraNoTraba]`	`notificarMantenimiento()`	`Fuera_De_Servicio_Mantenimiento`	Si la cerradura no traba, la puerta se retira de servicio.
`Alarma_Violacion_Seguridad`	`revisarIncidente`	—	`registrarIncidente()`	`Fuera_De_Servicio_Mantenimiento`	Tras la alarma, el hardware se revisa antes de volver a operar.
`Fuera_De_Servicio_Mantenimiento`	`completarMantenimiento`	—	`probarCerradura()`	`Vacio_Disponible`	Solo una puerta reparada vuelve a estar disponible.

Acciones internas: `Alarma_Violacion_Seguridad` ejecuta `entry / encenderLedRojo()`. El hardware opera en ciclo continuo, por lo que el diagrama no tiene estado final.

Análisis de casos de borde y resiliencia

Escenario	Cómo lo maneja el diagrama
Caídas de red	El diagrama no define un estado por falta de conectividad. Los eventos de sensores (peso, apertura

LO QUE HACES ¡IMPORTA!

Escenario	Cómo lo maneja el diagrama
	forzada) y el temporizador `after(72h)` se asumen locales al dispositivo.
Timeouts	`after(72h)` mueve el paquete no retirado a `Vencido_Pendiente_Recoleccion` para que un repartidor lo devuelva al hub.
Cancelaciones abruptas	Una apertura forzada sin comando digital lleva a `Alarma_Violacion_Seguridad`. Una falla de cerradura con la puerta abierta lleva a `Fuera_De_Servicio_Mantenimiento`.
Reintentos	No se define un límite de intentos de PIN. Un PIN inválido no cumple la guarda `[pinValido]` y el estado no cambia.

Observación para validar: el enunciado no define las salidas de `Alarma_Violacion_Seguridad`; se propone pasar por `Fuera_De_Servicio_Mantenimiento` antes de volver a `Vacio_Disponible`.

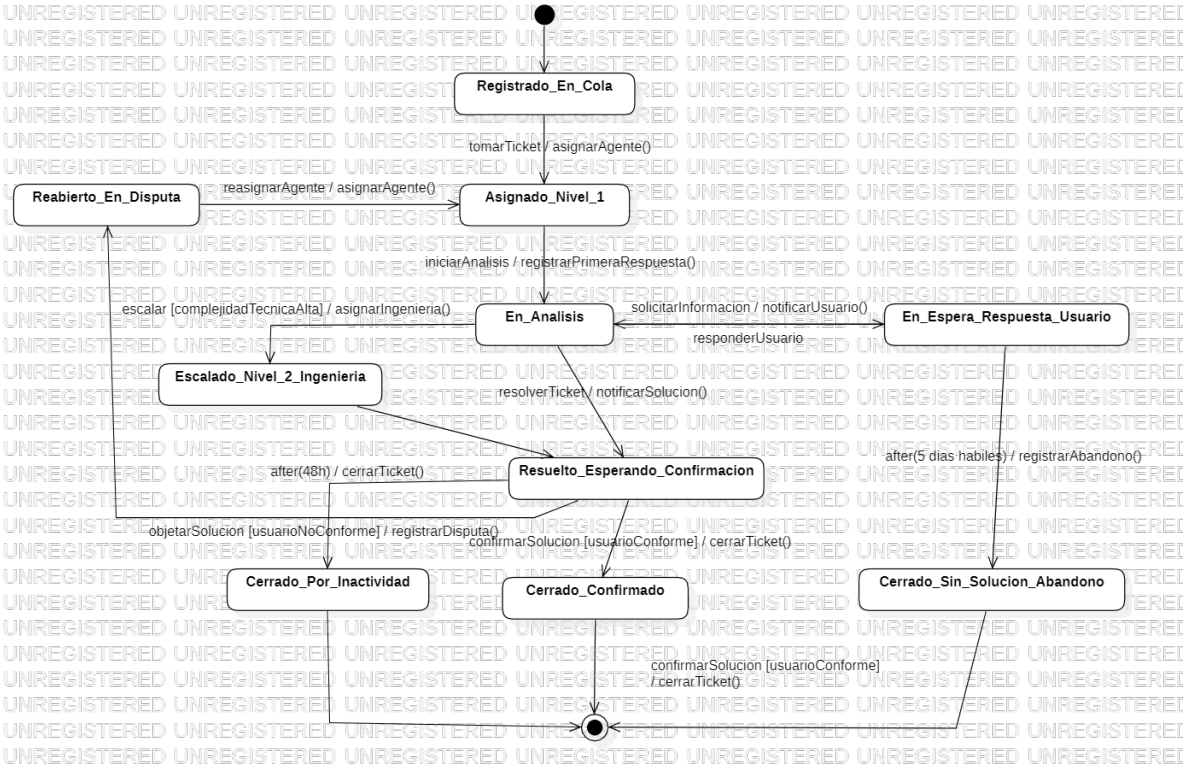
VII. Entidad 06: TicketDeSoporte

Objeto representado: incidencia abierta por un usuario ante reclamos o dudas. Controla los SLA de atención al cliente y los escalamientos N1 y N2.

Diagrama de estados

Figura 6. Máquina de estados de `TicketDeSoporte`.

LO QUE HACES ¡IMPORTA!



Matriz de transición de estados

Estado origen	Evento disparador	Condición de guarda	Efecto / acción	Estado destino	Justificación arquitectónica
Inicio	—	—	—	`Registrado_En_Cola`	El ticket nace al registrarse el reclamo.
`Registrado_En_Cola`	`tomarTicket`	—	`asignarAgente()`	`Asignado_Nivel_1`	Un agente de nivel 1 toma el ticket de la cola.

LO QUE HACES ¡IMPORTA!

Estado origen	Evento disparador	Condición de guarda	Efecto / acción	Estado destino	Justificación arquitectónica
Asignado_Nivel_1	iniciarAnalisis	—	registrarPrimeraRespuesta()	En_Analisis	El agente empieza a revisar el caso y queda registrada la primera respuesta para el control del SLA.
En_Analisis	solicitarInformacion	—	notificarUsuario()	En_Espera_Respuesta_Usuario	El agente pide datos adicionales al usuario.
En_Espera_Respuesta_Usuario	responderUsuario	—	—	En_Analisis	La respuesta del usuario reanuda el análisis.
En_Espera_Respuesta_Usuario	after(5 dias habiles)	—	registrarAbandono()	Cerrado_Sin_Solucion_Abandono	Si el usuario nunca responde, el ticket no queda abierto

Estado origen	Evento disparador	Condición de guarda	Efecto / acción	Estado destino	Justificación arquitectónica
					indefinidamente.
En_Analisis	escalar	[complejidadTecnicaAlta]	asignarIngenieria()	Escalado_Nivel_2_Ingenieria	Un problema técnico de alta complejidad pasa a ingeniería (N2).
En_Analisis	resolverTicket	—	notificarSolucion()	Resuelto_Esperando_Confirmacion	El agente propone la solución y pide confirmación al usuario.
Escalado_Nivel_2_Ingenieria	resolverTicket	—	notificarSolucion()	Resuelto_Esperando_Confirmacion	Ingeniería resuelve y también pide confirmación al usuario.
Resuelto_Esperando_Confirmacion	after(48h)	—	cerrarTicket()	Cerrado_Por_Inactividad	Sin confirmación ni objeción en 48 horas, el

Estado origen	Evento disparador	Condición de guarda	Efecto / acción	Estado destino	Justificación arquitectónica
					ticket se cierra automáticamente.
Resuelto_Esperando_Confirmacion	objetarSolucion	[usuarioNoConforme]	registrarDisputa()	Reabierto_En_Disputa	El usuario en desacuerdo reabre el caso en lugar de cerrarlo.
Resuelto_Esperando_Confirmacion	confirmarSolucion	[usuarioConforme]	cerrarTicket()	Cerrado_Confirmado	La confirmación expresa del usuario cierra el ticket.
Reabierto_En_Disputa	reasignarAgente	—	asignarAgente()	Asignado_Nivel_1	El caso reabierto vuelve a atención.
Cerrado_Por_Inactividad, Cerrado_Sin_Solucion_Abandono,	—	—	—	Final	Fin del ciclo de vida del ticket.

Estado origen	Evento disparador	Condición de guarda	Efecto / acción	Estado destino	Justificación arquitectónica
`Cerrado_Confirmado`					

Acciones internas: En_Analisis ejecuta do / contabilizarTiempoSLA() y Cerrado_Confirmado ejecuta entry / liberarColaAgente().}

Análisis de casos de borde y resiliencia

Escenario	Cómo lo maneja el diagrama
Caídas de red	Si el usuario pierde conexión y no responde, los temporizadores `after(5 dias habiles)` y `after(48h)` cierran el ticket. El diagrama no define un estado de red.
Timeouts	`after(48h)` en `Resuelto_Esperando_Confirmacion` cierra por inactividad; `after(5 dias habiles)` en `En_Espera_Respuesta_Usuario` cierra por abandono.
Cancelaciones abruptas	Si el usuario abandona, el ticket termina en `Cerrado_Sin_Solucion_Abandono`. Una objeción posterior a la solución lo reabre en `Reabierto_En_Disputa`.
Reintentos	El ciclo `En_Analisis` y `En_Espera_Respuesta_Usuario` se repite mientras el usuario responda; un ticket reabierto vuelve a `Asignado_Nivel_1`. El enunciado no limita el número de reaperturas.

Observaciones para validar con el equipo:

LO QUE HACES ¡IMPORTA!

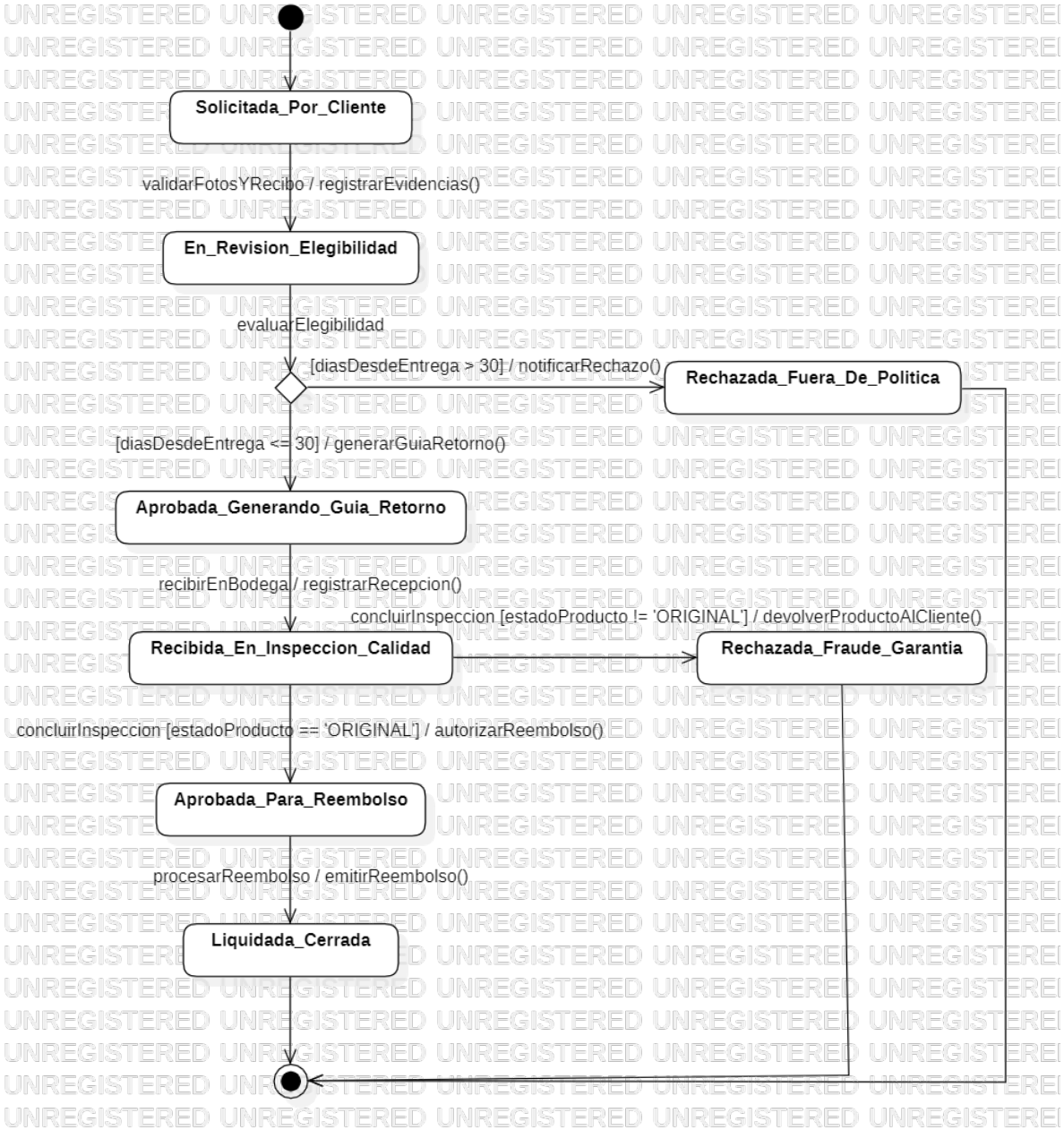
- El enunciado no nombra el estado de cierre cuando el usuario confirma la solución; se propone `Cerrado_Confirmado`.
- Tampoco define las salidas de `Reabierto_En_Disputa` ni de `Escalado_Nivel_2_Ingenieria`; se proponen `Asignado_Nivel_1` y `Resuelto_Esperando_Confirmacion`.

VIII. Entidad 07: SolicitudDeDevolucion

Objeto representado: proceso de devolución de mercadería y garantía posterior a la entrega (RMA o logística inversa). Cubre la inspección y el reembolso.

Diagrama de estados

Figura 7. Máquina de estados de `SolicitudDeDevolucion`.



UNREGISTERED

Matriz de transición de estados

Estado origen	Evento disparador	Condición de guarda	Efecto / acción	Estado destino	Justificación arquitectónica
Inicio	—	—	—	`Solicitada_Por_Cliente`	La devolución nace cuando el cliente la solicita.
`Solicitada_Por_Cliente`	`validarFotosYRecibo`	—	`registrarEvidencias()`	`En_Revision_Elegibilidad`	Atención valida fotos y recibo antes de decidir.
`En_Revision_Elegibilidad`	`evaluarElegibilidad`	—	—	Decisión C1	El plazo legal de 30 días se evalúa en un punto de decisión explícito.
Decisión C1	—	`[diasDesdeEntrega <= 30]`	`generarGuiaRetorno()`	`Aprobada_Generando_Guia_Retorno`	Dentro de la política de 30 días se habilita el retorno del producto.

Estado origen	Evento disparador	Condición de guarda	Efecto / acción	Estado destino	Justificación arquitectónica
Decisión C1	—	[diasDesdeEntrega > 30]	notificarRechazo()	Rechazada_Fuera_De_Politica	Pasados los 30 días legales, la solicitud se rechaza.
Aprobada_Generando_Guia_Retorno	recibirEnBodega	—	registrarRecepcion()	Recibida_En_Inspeccion_Calidad	El paquete llega a bodega y entra a inspección.
Recibida_En_Inspeccion_Calidad	concluirInspeccion	[estadoProducto == 'ORIGINAL']	autorizarReembolso()	Aprobada_Para_Reembolso	Solo un producto en estado original justifica el reembolso.
Recibida_En_Inspeccion_Calidad	concluirInspeccion	[estadoProducto != 'ORIGINAL']	devolverProductoAlCliente()	Rechazada_Fraude_Garantia	Un producto manipulado, dañado o sustituido se devuelve al cliente sin dinero.

Estado origen	Evento disparador	Condición de guarda	Efecto / acción	Estado destino	Justificación arquitectónica
`Aprobada_Para_Reembolso`	`procesarReembolso`	—	`emitirReembolso()`	`Liquidada_Cerrada`	El reembolso emitido cierra la solicitud.
`Liquidada_Cerrada`, `Rechazada_Fuera_De_Politica`, `Rechazada_Fraude_Garantia`	—	—	—	Final	Fin del ciclo de vida de la solicitud.

Acciones internas: Recibida_En_Inspeccion_Calidad ejecuta entry / registrarIngresoBodegaRMA() y Liquidada_Cerrada ejecuta exit / archivarExpedienteRMA().

Análisis de casos de borde y resiliencia

Escenario	Cómo lo maneja el diagrama
Caídas de red	Los avances dependen de eventos de atención y de bodega. Si una notificación se pierde, la solicitud permanece en su estado y no salta etapas. El diagrama no define un estado de red.

LO QUE HACES ¡IMPORTA!

Escenario	Cómo lo maneja el diagrama
Timeouts	No hay temporizadores. El plazo de 30 días es una guarda de elegibilidad evaluada en la decisión C1, no un evento temporal.
Cancelaciones abruptas	La cancelación de la solicitud por el cliente no está modelada. Una inspección que detecta alteración del producto termina en `Rechazada_Fraude_Garantia`, sin reembolso.
Reintentos	No hay reintentos: los estados rechazados son finales y una nueva devolución se registra como una solicitud nueva.

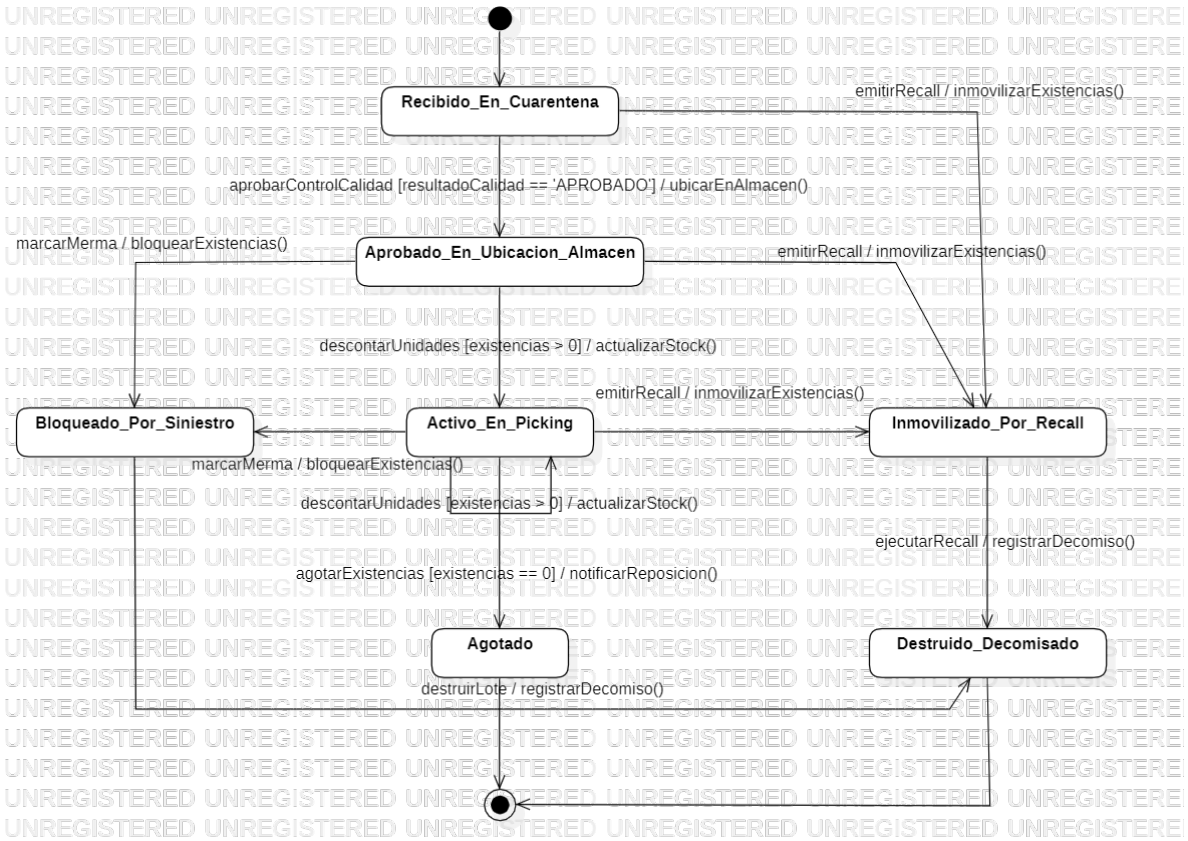
IX. Entidad 08: LoteDeInventario

Objeto representado: agrupación física de mercancías recibida de un proveedor. Cubre la cuarentena, el stock activo, las mermas y el bloqueo.

Diagrama de estados

Figura 8. Máquina de estados de `LoteDeInventario`.

LO QUE HACES ¡IMPORTA!



Matriz de transición de estados

Estado origen	Evento disparador	Condición de guarda	Efecto / acción	Estado destino	Justificación arquitectónica
Inicio	—	—	—	`Recibido_En_Cuarentena`	El lote entra al muelle de carga y queda aislado hasta el control de calidad.

LO QUE HACES ¡IMPORTA!

Estado origen	Evento disparador	Condición de guarda	Efecto / acción	Estado destino	Justificación arquitectónica
Recibido_En_Cuarentena	aprobarControlCalidad	[resultadoCalidad == 'APROBADO']	ubicarEnAlmacen()	Aprobado_En_Ubicacion_Almacen	Solo un lote aprobado por el control de calidad entra al stock.
Aprobado_En_Ubicacion_Almacen	descontarUnidades	[existencias > 0]	actualizarStock()	Activo_En_Picking	Los pedidos comienzan a descontar unidades del lote.
Activo_En_Picking	descontarUnidades	[existencias > 0]	actualizarStock()	Activo_En_Picking	Cada pedido descuenta unidades sin cambiar el estado del lote.
Activo_En_Picking	agotarExistencias	[existencias == 0]	notificarReposicion()	Agotado	Cuando las existencias llegan a cero el lote

Estado origen	Evento disparador	Condición de guarda	Efecto / acción	Estado destino	Justificación arquitectónica
					deja de ofrecerse.
`Aprobado_En_Ubicacion_Almacen`, `Activo_En_Picking`	`marcarMerma`	—	`bloquearExistencias()`	`Bloqueado_Por_Siniestro`	El supervisor bloquea el lote ante rotura de tubería, caducidad, plagas o caída de estantería.
`Bloqueado_Por_Siniestro`	`destruirLote`	—	`registrarDecomiso()`	`Destruido_Decomisado`	Desde el bloqueo solo cabe la destrucción o el decomiso.
`Recibido_En_Cuarentena`, `Aprobado_En_Ubicacion_Almacen`, `Activo_En_Picking`	`emitirRecall`	—	`inmovilizarExistencias()`	`Inmovilizado_Por_Recall`	Un recall del proveedor por defecto de fábrica inmoviliza el lote de inmediato.

Estado origen	Evento disparador	Condición de guarda	Efecto / acción	Estado destino	Justificación arquitectónica
`Inmovilizado_Por_Recall`	`ejecutarRecall`	—	`registrarDecomiso()`	`Destruido_Decomisado`	El lote retirado sale del inventario.
`Agotado`, `Destruido_Decomisado`	—	—	—	Final	Fin del ciclo de vida del lote.

Análisis de casos de borde y resiliencia

Escenario	Cómo lo maneja el diagrama
Caídas de red	Los eventos de inventario (descuentos, merma, recall) se aplican cuando llegan; hasta entonces el lote conserva su estado. El diagrama no define un estado de red.
Timeouts	No hay temporizadores. La caducidad se trata como una causa de siniestro, que el supervisor registra con `marcarMerma()`.
Cancelaciones abruptas	Un siniestro o un recall interrumpen la operación del lote: `marcarMerma` lleva a `Bloqueado_Por_Siniestro` y `emitirRecall` lleva a `Inmovilizado_Por_Recall`, desde varios estados.
Reintentos	No hay reintentos. `Bloqueado_Por_Siniestro` solo transiciona a `Destruido_Decomisado`, y un lote en recall no vuelve a picking.

Observación para validar: el enunciado no define cómo termina `Inmovilizado_Por_Recall` ni qué ocurre si el control de calidad rechaza el lote; se propone `ejecutarRecall` hacia `Destruido_Decomisado` para el primer caso.

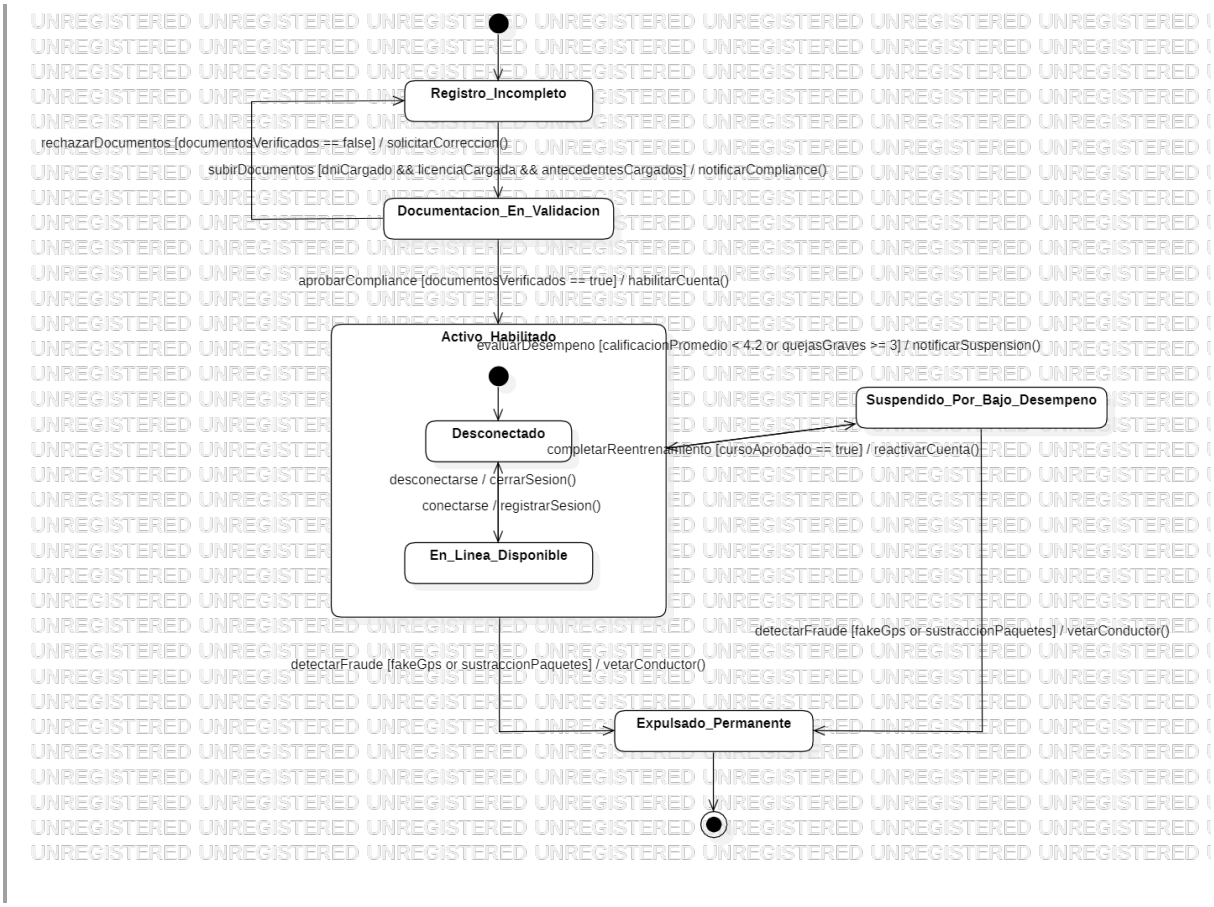
LO QUE HACES ¡IMPORTA!

X. Entidad 09: CuentaDeConductor

Objeto representado: perfil del repartidor (gig-worker). Modela su estado contractual y operativo en la plataforma: alta, suspensión y baja.

Diagrama de estados

Figura 9. Máquina de estados de `CuentaDeConductor`, con el estado compuesto `Activo_Habilitado` abierto.



Matriz de transición de estados

Estado origen	Evento disparador	Condición de guarda	Efecto / acción	Estado destino	Justificación arquitectónica
Inicio	—	—	—	`Registro_Incompleto`	La cuenta nace cuando el

Estado origen	Evento disparador	Condición de guarda	Efecto / acción	Estado destino	Justificación arquitectónica
					conductor inicia su registro.
`Registro_Incompleto`	`subirDocumentos`	`[dniCargado && licenciaCargada && antecedentesCargados]`	`notificarCompliance()`	`Documentacion_En_Validacion`	Compliance solo revisa expedientes completos.
`Documentacion_En_Validacion`	`aprobarCompliance`	`[documentosVerificados == true]`	`habilitarCuenta()`	`Activo_Habilitado`	Solo documentos verificados habilitan al conductor a operar.
`Documentacion_En_Validacion`	`rechazarDocumentos`	`[documentosVerificados == false]`	`solicitarCorreccion()`	`Registro_Incompleto`	El conductor corrige y vuelve a enviar sus documentos.
Inicio del compuesto	—	—	—	`Desconectado`	Al habilitarse, el conductor comienza

Estado origen	Evento disparador	Condición de guarda	Efecto / acción	Estado destino	Justificación arquitectónica
					fuera de línea.
Desconectado	conectarse	—	registrarSesion()	En_Linea_Disponible	El conductor queda disponible para recibir rutas.
En_Linea_Disponible	desconectarse	—	cerrarSesion()	Desconectado	El conductor deja de recibir rutas.
Activo_Habilitado (desde cualquier subestado)	evaluarDesempeno	[calificacionPromedio < 4.2 or quejasGraves >= 3]	notificarSuspension()	Suspendido_Por_Bajo_Desempeno	Un desempeño bajo suspende la cuenta sin importar si el conductor está conectado.
Suspendido_Por_Bajo_Desempeno	completarReentrenamiento	[cursoAprobado == true]	reactivarCuenta()	Activo_Habilitado	Solo el curso de reentrenamiento permite volver a operar.

Estado origen	Evento disparador	Condición de guarda	Efecto / acción	Estado destino	Justificación arquitectónica
`Activo_Habilitado` (desde cualquier subestado), `Suspendido_Por_Bajo_Desempeno`	`detectarFraude`	`[fakeGpsOrsustraccionPaquetes]`	`vetarConductor()`	`Expulsado_Permanente`	Manipular el GPS o sustraer paquetes expulsa al conductor de forma irreversible.
`Expulsado_Permanente`	—	—	—	Final	Fin del ciclo: la expulsión es irreversible.

Análisis de casos de borde y resiliencia

Escenario	Cómo lo maneja el diagrama
Caídas de red	Una pérdida de conexión del conductor se refleja moviéndolo a `Desconectado` con `desconectarse`; no hay un estado propio para la falta de señal. La manipulación de GPS se trata aparte, con `detectarFraude`.
Timeouts	El diagrama no define temporizadores.
Cancelaciones abruptas	Un fraude lleva a `Expulsado_Permanente`, de forma irreversible, desde `Activo_Habilitado` (cualquier subestado) o desde `Suspendido_Por_Bajo_Desempeno`.
Reintentos	Si Compliance rechaza los documentos, el conductor vuelve a `Registro_Incompleto`; tras una

LO QUE HACES ¡IMPORTA!

Escenario	Cómo lo maneja el diagrama
	suspensión, el reentrenamiento devuelve a `Activo_Habilitado`. Desde `Expulsado_Permanente` no hay retorno.

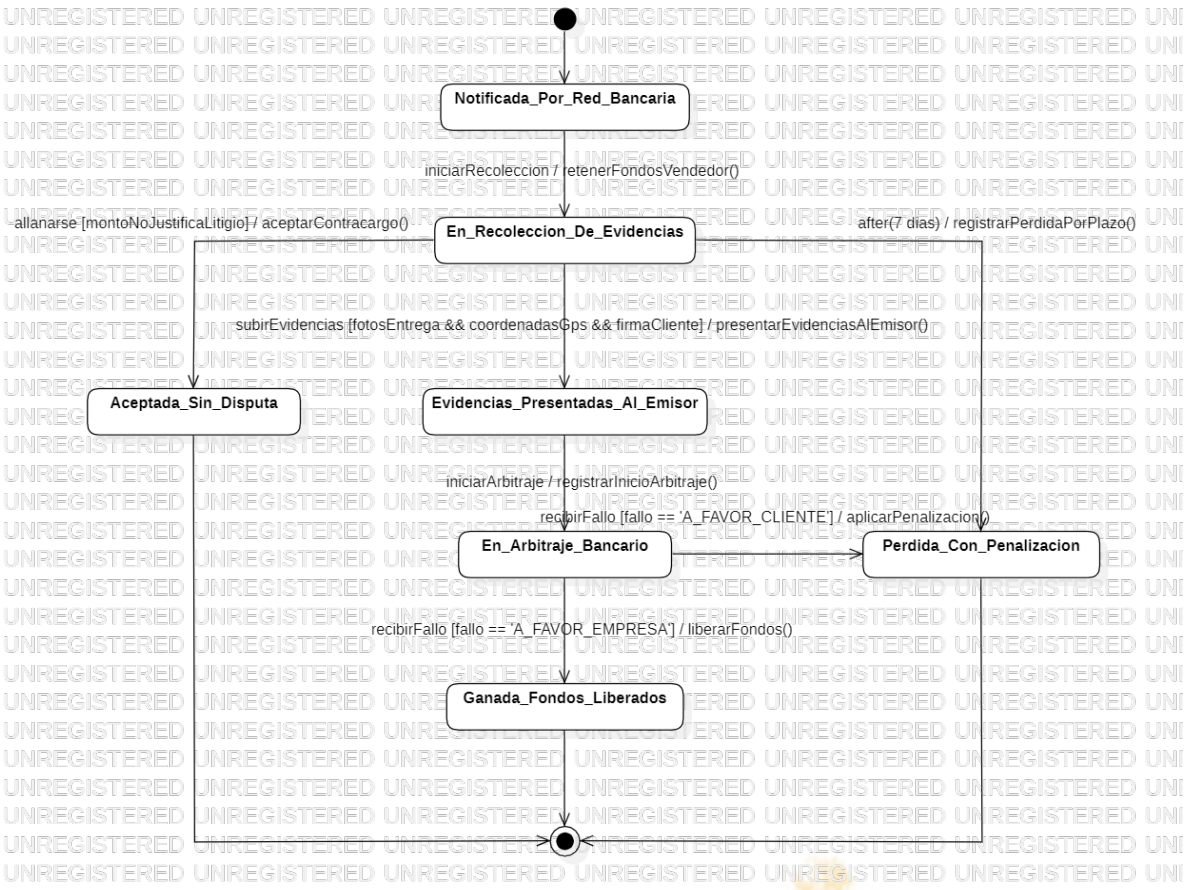
Observación para validar: el enunciado no define qué ocurre si Compliance rechaza los documentos ni cuál es el subestado inicial de `Activo_Habilitado`; se proponen `Registro_Incompleto` y `Desconectado`.

XI. Entidad 10: DisputaBancaria

Objeto representado: conflicto legal y financiero (chargeback o contracargo) iniciado por un tarjetahabiente, a través de su banco emisor, por fraude o no recepción. Cubre reclamos, evidencias y arbitraje.

Diagrama de estados

Figura 10. Máquina de estados de `DisputaBancaria`.



LO QUE HACES ¡IMPORTA!

Matriz de transición de estados

Estado origen	Evento disparador	Condición de guarda	Efecto / acción	Estado destino	Justificación arquitectónica
Inicio	—	—	—	`Notificada_Por_Red_Bancaria`	La disputa nace cuando la red bancaria notifica el contracargo.
`Notificada_Por_Red_Bancaria`	`iniciarRecoleccion`	—	`retenerFondosVendedor()`	`En_Recoleccion_De_Evidencias`	LogiFlow retiene los fondos del vendedor mientras se resuelve la disputa.
`En_Recoleccion_De_Evidencias`	`subirEvidencias`	`[fotosEntrega && coordenadasGps && firmaCliente]`	`presentarEvidenciasAlEmisor()`	`Evidencias_Presentadas_Al_Emisor`	El expediente de entrega (fotos, GPS y firma) respalda la defensa.
`En_Recoleccion_De_Evidencias`	`after(7 dias)`	—	`registrarPerdidaPorPlazo()`	`Perdida_Con_Penalizacion`	Sin evidencias antes del plazo límite de 7 días se

LO QUE HACES ¡IMPORTA!

Estado origen	Evento disparador	Condición de guarda	Efecto / acción	Estado destino	Justificación arquitectónica
					pierde la disputa.
En_Recoleccion_De_Evidencias	allanarse	[montoNoJustificaLitigio]	aceptarContracargo()	Aceptada_Sin_Disputa	La empresa evita el costo de litigar cuando el monto no lo justifica.
Evidencias_Presentadas_Al_Emisor	iniciarArbitraje	—	registrarInicioArbitraje()	En_Arbitraje_Bancario	El emisor evalúa las evidencias en un arbitraje de hasta 45 días.
En_Arbitraje_Bancario	recibirFallo	[fallo == 'A_FAVOR_EMPRESA']	liberarFondos()	Ganada_Fondos_Liberados	Un fallo favorable libera los fondos retenidos al vendedor.
En_Arbitraje_Bancario	recibirFallo	[fallo == 'A_FAVOR_CLIENTE']	aplicarPenalizacion()	Perdida_Con_Penalizacion	Un fallo en contra implica perder el

Estado origen	Evento disparador	Condición de guarda	Efecto / acción	Estado destino	Justificación arquitectónica
					monto y recibir una penalización.
`Ganada_Fondos_Liberados`, `Perdida_Con_Penalizacion`, `Aceptada_Sin_Disputa`	—	—	—	Final	Fin del ciclo de vida de la disputa.

Análisis de casos de borde y resiliencia

Escenario	Cómo lo maneja el diagrama
Caídas de red	Si falla la carga de evidencias, `after(7 dias)` evita que la disputa quede abierta: pasa a `Perdida_Con_Penalizacion`. El diagrama no define un estado de red.
Timeouts	`after(7 dias)` en `En_Recoleccion_De_Evidencias` fija el plazo límite. El arbitraje de hasta 45 días es una espera externa: no hay temporizador y la entidad aguarda `recibirFallo`.
Cancelaciones abruptas	`allanarse` permite abandonar la disputa y pasar a `Aceptada_Sin_Disputa` cuando el monto no justifica litigar.

LO QUE HACES ¡IMPORTA!

Escenario	Cómo lo maneja el diagrama
Reintentos	No hay reintentos: vencido el plazo de 7 días o emitido el fallo, los estados son finales.

Observación para validar: el enunciado no nombra el evento que inicia el arbitraje; se asume que comienza al presentarse las evidencias (`iniciarArbitraje`).

XII. Diseño de Persistencia y Auditoría de Estados (Script SQL)

Para garantizar la trazabilidad inmutable de cada cambio de estado en la base de datos relacional, se implementa la tabla **historial_cambio_estados**, la cual registra mediante claves foráneas y restricciones de integridad el estado de origen, el estado destino, el evento disparador, el usuario o proceso responsable y el contexto en formato JSON:

```
CREATE TABLE cat_entidades_logiflow (
    id_entidad_tipo INT PRIMARY KEY IDENTITY(1,1),
    nombre_entidad VARCHAR(60) NOT NULL UNIQUE,
    fecha_creacion DATETIME2 NOT NULL DEFAULT SYSDATETIME()
);

CREATE TABLE historial_cambio_estados (
    id_historial BIGINT PRIMARY KEY IDENTITY(1,1),
    id_entidad_tipo INT NOT NULL,
    id_registro_negocio VARCHAR(64) NOT NULL,
    estado_origen VARCHAR(80) NULL,
    estado_destino VARCHAR(80) NOT NULL,
    evento_disparador VARCHAR(100) NOT NULL,
    condicion_guarda VARCHAR(150) NULL,
    accion_ejecutada VARCHAR(150) NULL,
    usuario_responsable VARCHAR(80) NOT NULL,
    payload_json NVARCHAR(MAX) NULL,
    timestamp_transicion DATETIME2 NOT NULL DEFAULT SYSDATETIME(),
    CONSTRAINT FK_Historial_Entidad FOREIGN KEY (id_entidad_tipo)
        REFERENCES cat_entidades_logiflow(id_entidad_tipo),
    CONSTRAINT CHK_Transicion_Efectiva CHECK (estado_origen IS NULL OR estado_origen <>
estado_destino)
);

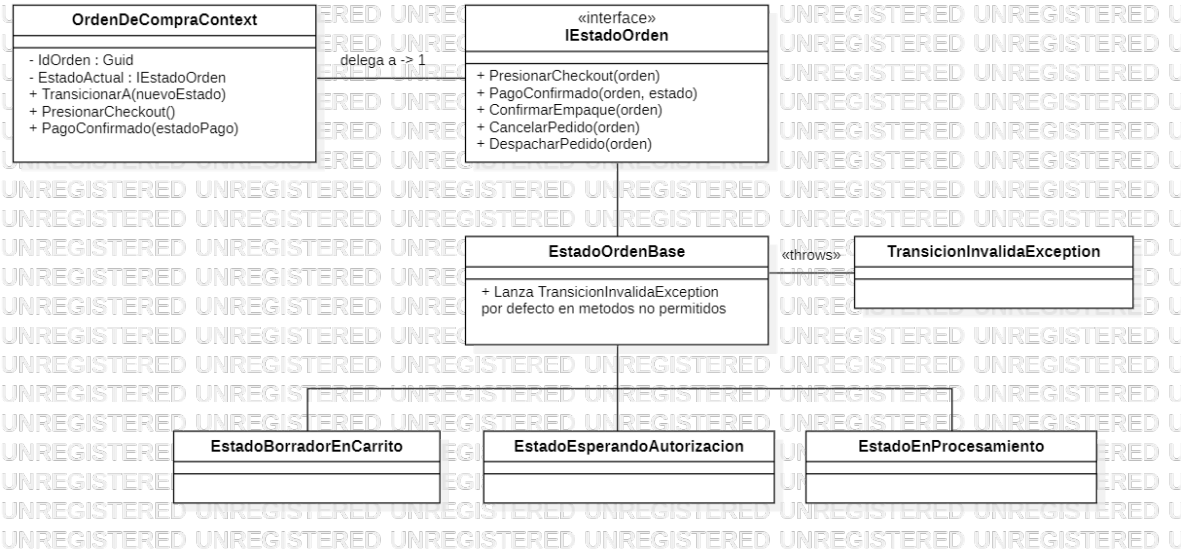
CREATE INDEX IX_Auditoria_Busqueda
```

LO QUE HACES ¡IMPORTA!

ON historial_cambio_estados (id_entidad_tipo, id_registro_negocio, timestamp_transicion DESC);

XIII. Propuesta de Implementación del Patrón State (Diagrama de Clases y Código Ilustrativo)

Figura 11. Diagrama de clases UML del Patrón de Diseño State aplicado a OrdenDeCompra




```
using System;

namespace LogiFlow360.Domain.StatePattern
{

    public class TransicionInvalidaException : InvalidOperationException
    {
        public TransicionInvalidaException(string estadoActual, string eventoIntentado)
            : base($"[ERROR DE DOMINIO] Transición rechazada: No se permite ejecutar el evento
'{eventoIntentado}' cuando la orden se encuentra en el estado '{estadoActual}'.")
        {
        }
    }


    public interface IEstadoOrden
    {
        string NombreEstado { get; }
        void PresionarCheckout(OrdenDeCompraContext orden);
        void PagoConfirmado(OrdenDeCompraContext orden, string estadoPago);
        void DetectarQuiebreStock(OrdenDeCompraContext orden, int stockDisponible, int cantidadPedida);
        void ConfirmarEmpaque(OrdenDeCompraContext orden);
        void CancelarPedido(OrdenDeCompraContext orden);
        void DespacharPedido(OrdenDeCompraContext orden);
        void ConfirmarEntrega(OrdenDeCompraContext orden);
        void ExpirarPorTimeout(OrdenDeCompraContext orden);
    }
```

LO QUE HACES ¡IMPORTA!

```
public abstract class EstadoOrdenBase : IEstadoOrden
{
    public abstract string NombreEstado { get; }

    public virtual void PresionarCheckout(OrdenDeCompraContext orden) =>
        throw new TransicionInvalidaException(NombreEstado, nameof(PresionarCheckout));

    public virtual void PagoConfirmado(OrdenDeCompraContext orden, string estadoPago) =>
        throw new TransicionInvalidaException(NombreEstado, nameof(PagoConfirmado));

    public virtual void DetectarQuiebreStock(OrdenDeCompraContext orden, int stockDisponible, int
cantidadPedida) =>
        throw new TransicionInvalidaException(NombreEstado, nameof(DetectarQuiebreStock));

    public virtual void ConfirmarEmpaque(OrdenDeCompraContext orden) =>
        throw new TransicionInvalidaException(NombreEstado, nameof(ConfirmarEmpaque));

    public virtual void CancelarPedido(OrdenDeCompraContext orden) =>
        throw new TransicionInvalidaException(NombreEstado, nameof(CancelarPedido));

    public virtual void DespacharPedido(OrdenDeCompraContext orden) =>
        throw new TransicionInvalidaException(NombreEstado, nameof(DespacharPedido));

    public virtual void ConfirmarEntrega(OrdenDeCompraContext orden) =>
        throw new TransicionInvalidaException(NombreEstado, nameof(ConfirmarEntrega));

    public virtual void ExpirarPorTimeout(OrdenDeCompraContext orden) =>
        throw new TransicionInvalidaException(NombreEstado, nameof(ExpirarPorTimeout));
}


public class OrdenDeCompraContext
{
    public Guid IdOrden { get; } = Guid.NewGuid();
    public IEstadoOrden EstadoActual { get; private set; }
    public bool Empacada { get; set; } = false;

    public OrdenDeCompraContext()
    {

        EstadoActual = new EstadoBorradorEnCarrito();
    }

    public void TransicionarA(IEstadoOrden nuevoEstado, string accionEjecutada)
    {
        string estadoOrigen = EstadoActual.NombreEstado;
        EstadoActual = nuevoEstado;
        Console.WriteLine($"[AUDITORÍA] Orden {IdOrden}: {estadoOrigen} -> {nuevoEstado.NombreEstado} |
Efecto: {accionEjecutada}");
    }


    public void PresionarCheckout() => EstadoActual.PresionarCheckout(this);
    public void PagoConfirmado(string estadoPago) => EstadoActual.PagoConfirmado(this, estadoPago);
    public void ConfirmarEmpaque() => EstadoActual.ConfirmarEmpaque(this);
    public void CancelarPedido() => EstadoActual.CancelarPedido(this);
    public void DespacharPedido() => EstadoActual.DespacharPedido(this);
    public void ConfirmarEntrega() => EstadoActual.ConfirmarEntrega(this);
    public void ExpirarPorTimeout() => EstadoActual.ExpirarPorTimeout(this);
}
```

```
public class EstadoBorradorEnCarrito : EstadoOrdenBase
{
    public override string NombreEstado => "Borrador_En_Carrito";

    public override void PresionarCheckout(OrdenDeCompraContext orden)
    {
        orden.TransicionarA(new EstadoEsperandoAutorizacionPago(), "reservarInventario()");
    }
}

public class EstadoEsperandoAutorizacionPago : EstadoOrdenBase
{
    public override string NombreEstado => "Esperando_Autorizacion_Pago";

    public override void PagoConfirmado(OrdenDeCompraContext orden, string estadoPago)
    {

        if (estadoPago != "AUTORIZADO")
            throw new TransicionInvalidaException(NombreEstado, $"PagoConfirmado con estado
'{estadoPago}'");

        orden.TransicionarA(new EstadoEnProcesamientoAlmacen(), "notificarAlmacen()");
    }

    public override void ExpirarPorTimeout(OrdenDeCompraContext orden)
    {

        orden.TransicionarA(new EstadoCanceladaPorTimeout(), "liberarInventario()");
    }
}

public class EstadoEnProcesamientoAlmacen : EstadoOrdenBase
{
    public override string NombreEstado => "En_Procesamiento_Almacen";

    public override void ConfirmarEmpaque(OrdenDeCompraContext orden)
    {
        orden.Empacada = true;
        orden.TransicionarA(new EstadoEmpacada(), "generarEtiqueta()");
    }

    public override void CancelarPedido(OrdenDeCompraContext orden)
    {

        if (!orden.Empacada)
            orden.TransicionarA(new EstadoCanceladaPorUsuario(), "reembolsarAutomatico()");
    }
}

public class EstadoEmpacada : EstadoOrdenBase
{
    public override string NombreEstado => "Empacada";

    public override void DespacharPedido(OrdenDeCompraContext orden)
    {
        orden.TransicionarA(new EstadoDespachadaEnRuta(), "registrarSalida()");
    }

}

public class EstadoDespachadaEnRuta : EstadoOrdenBase
{
```

```
        public override string NombreEstado => "Despachada_En_Ruta";

        public override void ConfirmarEntrega(OrdenDeCompraContext orden)
        {
            orden.TransicionarA(new EstadoCompletadaEntregada(), "cerrarOrden()");
        }
    }
    public class EstadoCompletadaEntregada : EstadoOrdenBase
    {
        public override string NombreEstado => "Completada_Entregada";
    }

    public class EstadoCanceladaPorTimeout : EstadoOrdenBase
    {
        public override string NombreEstado => "Cancelada_Por_Timeout";
    }

    public class EstadoCanceladaPorUsuario : EstadoOrdenBase
    {
        public override string NombreEstado => "Cancelada_Por_Usuario";
    }
}
```

XIV. Lecciones Aprendidas y Conclusiones del Análisis

1. Diferencia Técnica entre Modelar Flujos Funcionales (Actividades) vs. Ciclos de Vida de Objetos (Estados) Durante el diseño arquitectónico de **LogiFlow 360°**, se consolidó la distinción crítica entre un diagrama de actividades y un diagrama de máquina de estados UML 2.5. Mientras que los diagramas de actividades o de secuencia modelan procesos procedimentales en los que intervienen múltiples actores (por ejemplo, pasos de interfaz de usuario o flujos entre departamentos), el diagrama de estados se centra exclusivamente en **un único objeto de negocio** (OrdenDeCompra, PaqueteFisico, TicketDeSoporte, etc.) para gobernar las condiciones exactas en las que dicha entidad puede existir a través del tiempo.

2. Erradicación de Estados Inconsistentes mediante el Patrón State En el desarrollo de software empresarial, controlar el estado de una transacción mediante banderas booleanas sueltas (isPaid, isPacked, isCancelled) o estructuras condicionales anidadas genera vulnerabilidades graves, como órdenes cobradas dos veces, paquetes que saltan etapas de custodia o reembolsos autorizados sin inspección de calidad. La traducción directa del diagrama de estados hacia el

Patrón de Diseño State garantiza que cada estado sea una clase independiente que solo acepta los eventos válidos para ese momento y arroja una TransicionInvalidaException ante cualquier petición fuera de orden.

3. Resiliencia Operativa y Prevención de "Pozos Negros" (*Black Holes*) El uso riguroso de la sintaxis canónica Evento [Guarda] / Acción() combinada con eventos temporales explícitos (after(30 min) en reservas de carrito, after(48h) y after(5 dias habiles) en tickets de soporte, after(72h) en casilleros IoT y after(7 dias) en disputas bancarias) asegura que ninguna entidad quede atrapada indefinidamente en un estado intermedio ante caídas de red o abandono del usuario. De este modo, el modelo garantiza la liberación automática de stock y la trazabilidad completa para las auditorías financieras y operativas de la plataforma.